Программная инженерия

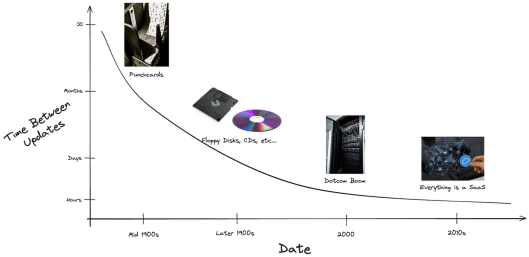
Лекция 11: Continious Integration и Continious Delivery

Денисов Артём Владимирович

Уральский Федеральный Университет

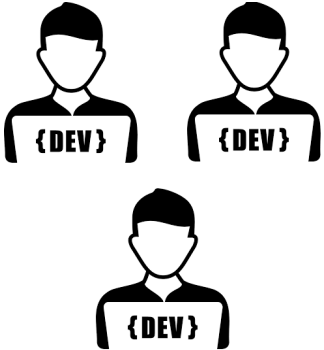
2026

Скорость доставки ПО со временем



Эпоха	Метод доставки	Время от написания до поставки
1960–70-е	**Перфокарты**	*Годы?*
1980–90-е	**Дискеты, CD диски**	Месяцы
2000-е	**Сервера**	Ежедневно–еженедельно
2010-е +	**CI/CD и облака**	Несколько раз в день

Представим традиционный подход



Командная разработка

- Несколько разработчиков, параллельные ветки
- Каждый работает изолированно
- Синхронизация прогресса редкая

Что происходит при редкой интеграции

Что самое неприятное в работе программиста?

Что происходит при редкой интеграции

Что самое неприятное в работе программиста?

Merge конфликты

- **Merge hell**: слияние изменений в коде становится настолько сложным, что превращается в отдельную проблему: с конфликтами, ошибками и потерей времени.

Проблема в Git?

Симптом

- Конфликты слияния
- Долгая интеграция
- Позднее распознавание багов

Проблема в Git?

Симптом

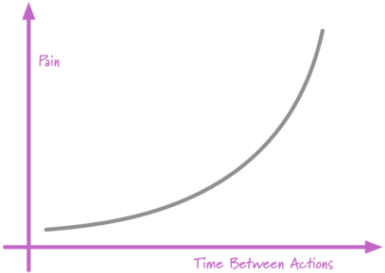
- Конфликты слияния
- Долгая интеграция
- Позднее распознавание багов

Причина

- Редкая интеграция
- Большие порции изменений
- Организация работы

Проблема — в процессе, не в инструменте.

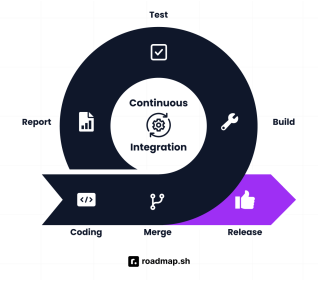
Ключевой принцип



«If it hurts — do it often,
and it will not hurt so much»

— ***Martin Fowler***

Решение - Continuous Integration



Continuous Integration (CI)

Практика, при которой разработчики часто интегрируют код в общую ветку, с автоматической проверкой каждой интеграции.

Почему CI работает

- Маленькие изменения — дешёвые конфликты
- Обратная связь за минуты, не за недели
- Ошибки ловятся возле контекста

Внедрили CI - появились проблемы

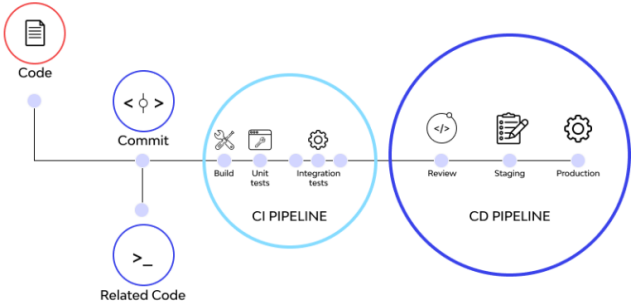
Проблемы

- Частые коммиты - неполный функционал
- Сборка падает чаще
- Много багов
- Основная ветка нестабильна

Решение

- Ввести строгие правила и проверки принятия изменений

Типичный CI-пайплайн



- Последовательность автоматических шагов
- Каждый коммит проходит весь путь

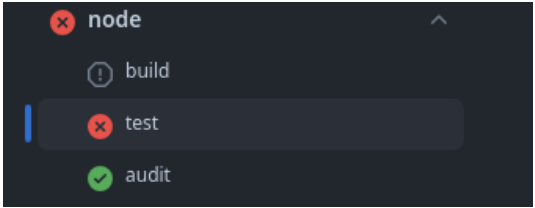
CI: Термины

- Pipeline — это упорядоченная последовательность автоматических шагов, через которые проходит каждое изменение кода.
- Триггер — событие, запускающее пайплайн: push в ветку, открытие pull request, merge, расписание.
- Этап сборки (build): код компилируется или упаковывается в выполнимую форму.
- Линтинг и статический анализ: автоматическая проверка стиля и потенциальных дефектов без запуска кода.

CI: Тесты и артефакты

- Unit-тесты: проверка отдельных функций и модулей в изоляции.
- Integration-тесты: проверка взаимодействия компонентов между собой.
- Сборка артефактов: создание воспроизводимых единиц поставки (jar-файл, docker-образ, бинарник).

Принцип fail-fast



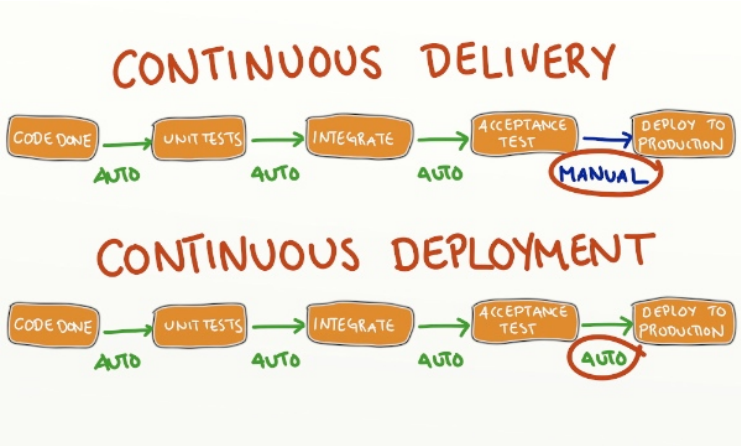
- Дешёвые проверки — раньше дорогих
- Упало — дальше не идём
- Быстрая обратная связь

Что дальше?

- Ручные деплои = ошибки и человеческий фактор
- Редкие релизы = большие риски

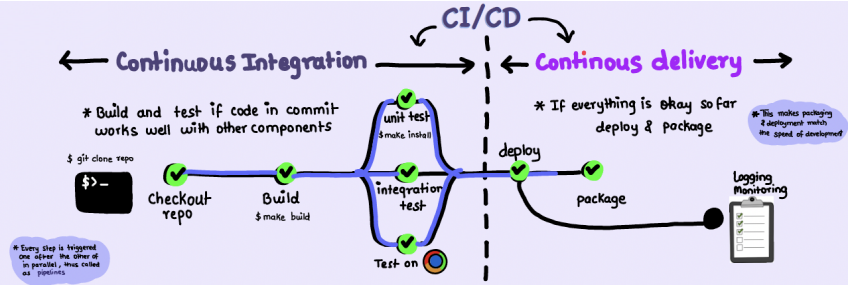
Нужна автоматизация пути новых изменений до прода

CI и CD пайплайн

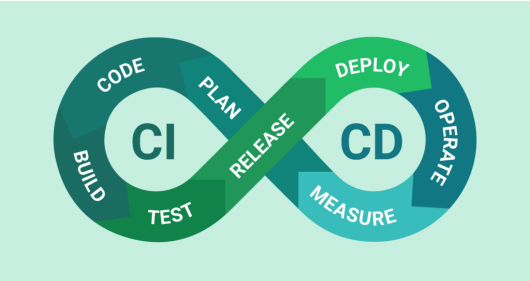


CD дополняет CI

CI и CD. Где граница?



Continuous Delivery



Смысл Continuous Delivery

Любая версия, прошедшая CI, автоматически доводится до состояния, готового к развёртыванию в production.

Ценность Continuous Delivery

- Релиз — рутина, а не праздник
- Мелкие релизы $\Rightarrow$ локализуемые ошибки
- Тривиальный откат

Путь изменений: Delivery Pipeline

$$\texttt{Commit} \to \texttt{Build} \to \texttt{Test} \to \texttt{Artifact}$$
$$\to \texttt{Staging} \to \texttt{Gates} \to \texttt{Production}$$

- Единый конвейер
- Один артефакт на всех стадиях

Delivery vs Deployment

Continuous Delivery

- Готов к релизу в любой момент
- Финальное решение — за человеком

Continuous Deployment

- Релиз идёт автоматически

Что выбирать

Delivery vs Deployment

- Delivery подходит практически всем
- Deployment — для зрелых команд
- Выбор определяет цена ошибки
- Это организационное решение, не техническое

Инструменты CI/CD

- Jenkins
- GitLab CI/CD
- GitHub Actions
- …

Jenkins

- Self-hosted, максимально гибкий
- Огромная экосистема плагинов
- Требует поддержки и обслуживания
- Уместен: enterprise, закрытый контур

GitLab CI/CD

- Часть платформы GitLab
- Единая экосистема: repo + CI + registry
- Облако или self-hosted
- Уместен: если проект уже на GitLab

GitHub Actions

GitHub Actions

- Облачный, встроен в GitHub
- Низкий порог входа, маркетплейс workflow
- Привязка к экосистеме GitHub
- Уместен: если проект уже на GitLab, OSS

Скорость pipeline — это важно

Пайплайны должны быть быстрыми

- Медленный pipeline игнорируют
- Первая обратная связь — за минуты
- Ускорение важнее новых проверок

Резюме

CI/CD

- Проблема: редкая интеграция, рискованные релизы
- CI: частая интеграция + автопроверка
- CD: автоматический путь до релиза

Практика. Github Actions

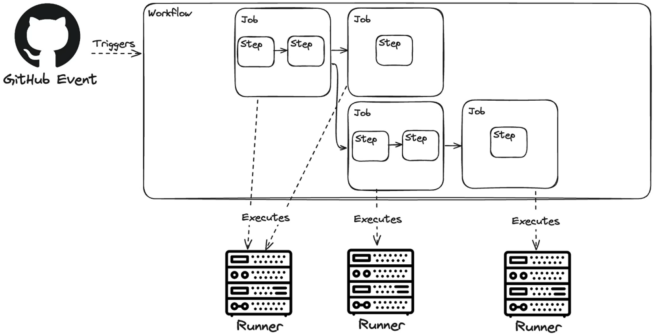
Что такое Github Actions

Определение

Github Actions - платформа для автоматизации пользовательских workflow.

CI/CD пайплайн - один из пользовательских workfkow, но не единственный

Github Actions концепции



Event-driven автоматизация

Events

Event это

Когда что-то случается с репозиторием изнутри или извне

Примеры

- Pull request создан
- Pull request слит в master ветку
- Push в ветку
- Issue создан

Actions

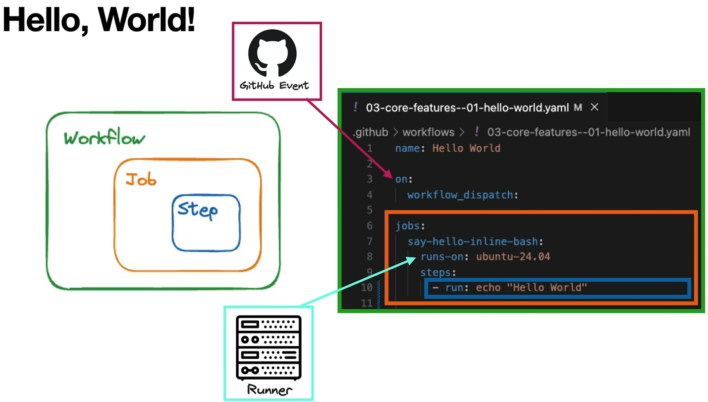
Action это

Действие в ответ на событие

Примеры

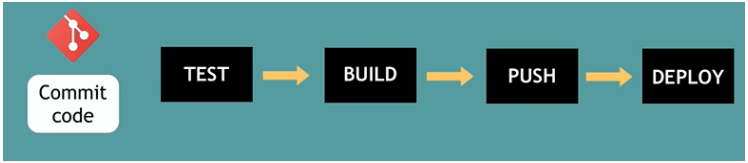
- Issue создан (Event) -> На Issue назначается label (Action) и assignee (Action)
- Pull request в master (Event) -> Прогоняются автоматические тесты(Action)

Пример автоматизации



Workflow как код (YAML)

CI/CD в Github Actions



CI/CD пайплайн

Источники информации и изображений

1. https://www.youtube.com/watch?v=Xwpi0ITkL3U
2. https://github.com/sidpalas/devops-directive-github-actions-course
3. https://roadmap.sh/devops/lifecycle
4. https://martinfowler.com/bliki/FrequencyReducesDifficulty.html
5. https://bytebytego.com/guides/cicd-pipeline-explained-in-simple-terms/
6. https://micoope.com.gt/?o=what-is-ci-cd-pipeline-explained-with-benefits-nn-xWxja2F5
7. https://blog.xpeducacao.com.br/pipeline-ci-cd/
8. https://blog.crisp.se/2013/02/05/yassalsundman/continuous-delivery-vs-continuous-deployment
9. https://docs.github.com/ru/actions
10. https://www.youtube.com/watch?v=R8_veQiYBjI
11. https://stackoverflow.com/questions/78293605/disabling-fail-fast-on-workflow-call

Photographs used under public domain or fair usage. No copyright infringement intended. If used accidently under wrong license please contact to be removed.

Программная инженерия

Лекция 11: Continious Integration и Continious Delivery

Денисов Артём Владимирович

Уральский Федеральный Университет

2026